SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

A. Method and Training Details

A.1. Error Decomposition

This subsection derives Eq. 2 and the rotation and context terms behind the task groups of Appendix B.1, and shows how the error of an estimate scales with the lead and the motion.

Translation. Write the true target position at the execution time as $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, with the velocity $\mathbf{v}$ at τ_t and the remaining motion $\boldsymbol{\rho}$, and the policy's estimate as $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S1}$$

and the triangle inequality yields Eq. 2 with $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$ and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. The bound holds for any split of the estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$, and e_{dyn} absorbs the higher-order part of the timing error. The residual $\boldsymbol{\rho}$ is the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, which an acceleration estimate from the history reduces; impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce.

Rotation and the contact point. Write the true orientation as $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$, with the angular velocity $\boldsymbol{\omega}$ and the remaining rotation $\boldsymbol{\eta}$, and its estimate likewise. The rotation angle is bi-invariant, and Exp is 1-Lipschitz from $\mathbb{R}^3$ to SO(3), so for general rotations

$$\begin{aligned}\theta &= \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \\ &\le \theta_{\mathrm{loc}} + \|\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}} - \boldsymbol{\omega}\Delta - \boldsymbol{\eta}\| \\ &\le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta,\end{aligned} \tag{S2}$$

with the current orientation error θ_{loc}, $e_\omega = \|\hat{\boldsymbol{\omega}} - \boldsymbol{\omega}\|$, and $e_\eta = \|\hat{\boldsymbol{\eta}} - \boldsymbol{\eta}\|$. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, is off by $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$ at τ^{a}, and the operator norm of $\hat{R}R^{\top} - I$ is $2\sin(\theta/2)$, so its error is

$$e_{\mathrm{c}} \le e + 2r\sin(\theta/2) \le e + r\theta, \tag{S3}$$

with $e = \|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\|$. A grasp or placement constrains several points spread over the body, such as the two finger contacts or the arms of a T-shaped part, so no single tracked point removes the term $r\theta$: the orientation error enters the contact scaled by the lever arm, and the target needs its full SE(3) state.

Context. Let m be the largest margin that a feasible path can keep between the robot and the carried object and the context it passes, measured to the predicted positions of the context at τ^{a}. If the executed path keeps this margin, a collision requires a context error of at least m, so a coarse estimate of the surroundings suffices when $m \gg \varepsilon$; the no-scene-flow control of Sec. 4.3 tests this coarse supervision directly. A driver, such as a conveyor, a ramp, or a striker, acts on the contact through the motion it imparts to the target, which enters Eq. 2 through the target's own terms. The SE(3) history and forecast represent this motion precisely, so the driver needs no precise state of its own.

A.2. Architecture

Table S1 lists the configuration of every module, in the order of the data flow described below.

Scene tokens. The frozen MAE ViT-B/16 [4] encodes the agent-view RGB into 196 patch tokens; we selected it on development scenes (Appendix E.1). Each patch token adds a learned view embedding, shared by all patches in our single-view setting, and linear embeddings of its mean valid metric depth and of the base-frame position of its back-projected center pixel. These positions also drive 3D rotary embeddings of the patch queries and keys, which no other token receives, and the same center points are the source points of the sparse 3D flow. Past images never enter the model.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3)$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities from interval displacements over the elapsed time and accelerations

Table S1. Architecture of SE(3)-WAM. Heads are skipped at deployment.

Module	Configuration
Visual encoder	frozen MAE ViT-B/16, 224×224 input, 196 patch tokens
Target-state tokenizer	$28 \to 1024 \to 1024$ GELU MLP, additive relative-time embedding
Robot-state tokenizer	linear, $7 \to 1024$
Tokens	212 scene (196 patch, 8 target-state, 8 robot), 16 summary, 8 action
Backbone	24 layers, width 1024, 16 heads, shared attention, SiLU FFN 4096 per stream, 504.0M parameters
Pose head	linear, mean summary state $\to 8 \times 6$
Flow head	linear per patch token, $8 \times (3{+}1)$
Appearance heads	linear per patch token: depth and moving-surface RGB as 16×16 pixels, 1024-d DINOv3 ViT-L/16 features
Target reconstruction	$1024 \to 256 \to 3072$ GELU MLP with sigmoid, 32×32 RGB
Action stream	flow matching, adaptive layer norm on the flow time, 5 Euler steps

from adjacent velocities over the spacing of the interval centers, and slot 7 holds the current absolute pose in the base frame. We standardize each dimension with training statistics, zero invalid dimensions, and append three validity flags (pose, interval, and acceleration); a slot without any valid feature is masked out of attention. On the real robot, the tracker updates the pose at every control-period boundary (Appendix D.2), so the history keeps the same 50 ms spacing. The robot tokens hold the end-effector position, rotation vector, and measured gripper opening at the same offsets, and 16 learned embeddings form the summary tokens. The policy has no language token.

Backbone and mask. Each layer applies one attention over all tokens with query, key, value, and output projections shared by both streams, followed by separate SiLU feed-forward networks for the world and the action tokens. The mask of Sec. 3.3 gives a blocked edge the logit $-\infty$ in every layer and head, in training and at deployment, so the world states are computed once per query and reused across the Euler steps.

Forecast and auxiliary heads. All heads are readouts of the final world states, trained jointly with the action stream and skipped at deployment; no token reads their outputs. The pose head reads the mean summary state and is trained with $\mathcal{L}_P$ (Eq. 4). The flow head gives each patch point's displacement and visibility logit at every horizon, trained with $\mathcal{L}_F$ (Eq. 5) and the scale $s = 0.7154605$ m; the visibility loss has weight 1 in simulation, and points that are valid but invisible still supervise their displacement. The appearance heads predict at every horizon the metric depth with a squared error on valid pixels, the RGB of surfaces that move by more than 1 mm, including the robot, with a squared error, and the DINOv3 features [11] with a squared error after channel-wise layer normalization. The target-reconstruction head recovers the current target's RGB region from the mean summary state with an absolute error. These four losses form $\mathcal{L}_{\text{aux}}$.

Action stream. Each of the 8 action tokens embeds a seven-dimensional command, a base-frame translation increment, a left-relative rotation vector, and a gripper command; the increments refer to the measured end-effector pose at application. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each axis to 0.05 m and 0.5 rad; the gripper follows the sign of its command. Appendix B.3 specifies how the chunks are executed.

A.3. Training on Dynamic-14

Each control step of a demonstration anchors one window of 8 query positions with their own causal histories and no attention across positions, which gives 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. For the flow labels, each patch-center pixel is back-projected with the rendered depth and attached to the local frame of the MuJoCo geometry rendered at it; the ground-truth rigid transforms of that geometry carry the same surface point to the future horizons, which gives its cumulative displacement in the base frame. A future point is visible if it lies in front of the camera and inside the image and its projected pixel renders the same geometry at a depth within $\max(1\,\text{mm},\ 0.001z)$ of the point's depth z. Command j of the chunk for query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. 1), and commands are normalized with the mean and standard deviation of the valid commands of the training split. Each query position draws 8 noise–time pairs for its chunk and averages their losses, which reduces the variance of the action gradient without enlarging the batch, and each loss is normalized by its count of valid labels in the global batch. The DINOv3 features of future frames are cached once per episode. Table S2 lists the recipe.

A.4. Pretraining on DROID and Fine-Tuning

Data. The real-robot base trains on DROID [6], recorded at 15 Hz, with the 3D point tracks and camera calibrations that PointWorld released for it [5], matched to the raw episodes by exact agreement of robot states, commands, and camera identity. It uses the first exterior camera with its depth. Of

Table S2. Training hyperparameters on Dynamic-14.

Item	Value
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Action noise	8 noise–time pairs per query position
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

42,935 matched episodes, 42,087 yield 598,889 anchor windows, 538,740 from 37,874 training and 60,149 from 4,213 validation episodes. Each anchor expands into 8 consecutive query positions; the anchor carries the flow and DINOv3 labels, and the others carry action labels only. All times are capture timestamps, and the flow and DINOv3 targets keep PointWorld's record times. We keep the 96.88% of action slots with an issued command, without a filter on speed, so that the base is not biased toward slow motion.

Objectives. DROID provides no object poses, so the base has no target-state tokens or pose head. It reads the current RGB-D frame and the robot history (204 scene tokens) and trains the action loss and the sparse 3D flow with weight 1 and the future DINOv3 features with weight 0.1; PointWorld records only visible positions, so the visibility loss is off. Most DROID objects move only when the robot moves them, so the base mainly learns metric geometry and the motion caused by the robot's own actions; the exogenous motion of our tasks and the target pose enter during fine-tuning.

Flow labels. Each patch of the 14×14 grid follows one PointWorld track: among the tracks that start inside the patch, are initially visible, and have a valid depth, the one with the largest image motion consistent with its 3D displacement, and otherwise the one nearest to the patch center, so that moving objects missed by a fixed center point are covered. The label is the track's displacement from its start; 47.05% of the point–horizon slots carry a label.

Recipe and fine-tuning. The base has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch of 512 on 64 GPUs, with the optimizer and precision of Table S2. Each real-robot policy starts from the base and is fine-tuned on the demonstrations of its task [updates, learning rate, and batch size]; the target-state tokens and the heads that the base lacks are freshly initialized. No real-robot policy uses simulation weights, and the base sees no simulation data.

B. Dynamic-14 and Evaluation Protocol

B.1. Benchmark Design

Dynamic-14 comprises fourteen tasks in five families (Fig. 4; Sec. 4.1), which together cover translation, rotation, temporary occlusion, constrained passage, and abrupt changes of the target's motion. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Environment. The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [12, 18] with a Franka Panda arm and a parallel gripper, driven by robosuite's operational-space pose controller at a control period of 50 ms and a physics step of 2 ms. A fixed third-person agent-view camera at $(0.68, -0.58, 1.62)$ m with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector increment in the robot base frame, a translation and a rotation vector, together with a gripper command; the shared action adapter clips each command to 0.10 m per translation axis and 0.5 rad per rotation-vector component.

Success and termination. Table S3 summarizes the success condition of each task. We check them at every physics step, an interrupted continuous condition resets its counter, and an episode ends at its first terminal event: success, a failure such as unsafe contact, leaving the workspace, or a drop, or the time limit.

Roles and task groups. Table S3 names the target of each task. A moving receptacle is the target, since its pose decides containment; in the passage tasks, the wire entry, which the wand must match within 30 mm, is the target, and the rest of the wire is context that no contact may touch. The two task groups of Fig. 5 were fixed before the ablations from the terms of Appendix A.1: a task is rotation-sensitive ($r\theta$) if its target rotates during the lead and its orientation matters at contact, which excludes the balls, and context-active if it has an obstacle clearance m or a driver (a conveyor, a ramp, or a striker).

B.2. Scenarios, Demonstrations, and Splits

Scenarios. Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges, and discrete choices with equal probability: besides the speeds of Table S3, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15° with friction set

Table S3. **The fourteen Dynamic-14 tasks.** Target: the body whose pose the policy receives (Sec. 3.1). Speed: target speed range (capture tasks: at the ramp exit). Every task requires its stages in order without a terminal failure; a grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a 0.5 s hold. Limit: episode time limit. Groups of Fig. 5: R, rotation-sensitive; C, context-active (Appendix B.1). The released task configurations contain the exact thresholds and scenario ranges.

Task (code)	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp (grasp)	block	0.03–0.20	Grasp the block moving harmonically and yawing, lift, and hold.	8	R
Conveyor grasp (t1_belt)	part	0.04–0.10	Grasp the part on the conveyor, lift, and hold.	8	C
Translating grasp (t1_xyz)	part	0.04–0.15	Grasp the translating part, lift, and hold.	8	–
Rotating grasp (t1_so3)	part	0.04–0.15	Grasp the translating and rotating part ($\|\omega\| \leq 1.2$ rad/s), lift, and hold.	8	R
Occluded grasp (t1_occ)	part	0.04–0.10	Grasp the part on the conveyor, which a canopy hides for 0.5–3.0 s, without unsafe contact with the canopy; lift and hold.	8	C
Moving receptacle (t2_trans)	receptacle	0.03–0.06	Place the held object fully inside the moving receptacle and release it to rest.	10	–
Rotating receptacle (t2_se3)	T-shaped receptacle	0.03–0.06	Place the held T-shaped part fully inside the translating and rotating T-shaped receptacle and release it to rest.	10	R
Wire passage (t3_phase)	wire entry	[X]	Align the held wand's loop with the rotating wire within 30 mm, then pass it along the wire to the end without contact.	21	R, C
Full wire passage (t3_full)	wire entry	[X]	Grasp the wand, complete the passage, exit the wire, and release the wand.	32	R, C
Ball capture (t4_sphere)	ball	[0.4–0.7]	Catch the ball after it leaves the ramp and before it touches the tray, lift, and hold.	6	C
Ball capture on table (t4_sphere_tabletop)	ball	[0.4–0.7]	Grasp the ball while it still rolls on the table after the ramp, lift, and hold.	6	C
Slider capture (t4_slider)	slider	[0.4–0.8]	Grasp the block while it still slides on the table after the ramp, lift, and hold.	6	C
Can capture (t4_can)	can	[0.4–0.7]	Grasp the can while it still rolls on the table after the ramp, before it tips over, lift, and hold.	6	R, C
Post-impact grasp (t5_replan)	part	0.12–0.14	After a striker deflects the part on the conveyor, grasp it along its grasp axis, lift, and hold.	6	R, C

per object. An occluded scene is accepted only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames. We keep the scenes that pass physical-feasibility checks and replay the stored accepted scenes at evaluation.

Demonstrations. The expert is a task-specific closed-loop state machine with privileged simulator state: the target's current pose and linear and angular velocity, the robot state, the task geometry, and true contact, grasp, and stage events. At every control period it, for example, extrapolates the target over a short horizon and tracks, descends, closes, and lifts in the grasping tasks, predicts the interception point of a rolling or flying object under gravity in the capture tasks, and follows the known wire path with phase compensation from the current angle and angular velocity in the passage tasks. Its commands are clipped to 0.05 m and 0.5 rad per axis. A demonstration is accepted if its scene is feasible, the expert succeeds, an independent replay of the recorded actions reproduces the episode, and its path, action changes, and jerk pass numerical quality checks. The training pool of 11,200 demonstrations, 800 per task, is drawn from the accepted ones. We record them at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry; the learned policies receive only the inputs of Sec. 3.

Splits. We select configurations on 1,400 development scenes (100 per task). All Dynamic-14 results use a separate held-out set of 1,400 scenes, generated with the same ranges and new source seeds that appear in no training, validation, or development episode; the first 100 feasible scenes in increasing seed order form each task's set, which all methods share.

B.3. Real-Time Evaluation

Executor. All methods share the executor of Sec. 3.4 on the 50 ms control clock; the total latency L_t of a query is its inference latency under the protocol plus any added latency. Four rules complete Sec. 3.4. A method's chunk holds its native number H_m of commands (Table S4), and a chunk with $k_t \geq H_m$ has expired entirely and is dropped. With $L_t = 0$, the next query starts one control period after capture. Before the first chunk of an episode arrives, and whenever a

chunk runs out, the controller holds the current arm target and gripper command while physics advances; no command is repeated. The post-impact grasp applies each command 50 ms after it is issued ($d_{\text{task}} = 50$ ms in Eq. 1); all other tasks have $d_{\text{task}} = 0$.

Protocols. Under the real-time protocol, physics advances during each query by the method's native latency, drawn per query from its latency distribution measured on one RTX 5090. With paused physics, the simulation waits for inference; with equal latency, every method is charged the p95 latency of SE(3)-WAM; with original executors, each baseline keeps the execution scheme of its paper, such as temporal aggregation for ACT or streaming for DynamicVLA, at its native latency. The added-latency sweep of Table S7 adds 0, 50, 100, 150, or 200 ms after inference under the real-time protocol, and physics advances during it. The native-latency rollouts of ACT and Diffusion Policy ran on RTX 5090 and all other rollouts on RTX PRO 6000; every rollout replays the latency measured on RTX 5090, so the rollout hardware does not affect the results.

Latency measurement. We time all methods on one RTX 5090 (32 GB), the GPU of our robot, with exclusive use and batch size 1. Each method runs three rounds of thirty warm-up and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and a rotated method order, in the precision of its release; SE(3)-WAM uses FP32. A timed query starts from the newest observation, with the history already maintained, and ends when its first command is available. It includes all computation that needs the newest frame: preprocessing, image encoding, the online flow updates and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. For SE(3)-WAM, we time the forecast heads separately, although deployment skips them. This time-to-action is reported without rounding; the handover time $\ell_t = k_t \Delta_a$ rounds it up to the control period, and the action age of a command is its execution time minus the capture time. A query misses its deadline when its time-to-action exceeds 50 ms. Appendix C.2 reports the measured values.

B.4. Metrics and Statistics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{100}{14} \sum_{k=1}^{14} \frac{1}{S_m n_k} \sum_{s=1}^{S_m} \sum_{i=1}^{n_k} y_{mski}, \qquad \text{(S4)}$$

for binary outcomes y_{mski} of method m, trained with seed s, on scene i of task k, with $S_m = 3$ training seeds for our variants and $S_m = 1$ for the baselines. The *position error at first contact* e_p is the distance of the end effector from the expert's contact frame on the target. In the two placement tasks, which start with the object held, the contact is that of the held object with the receptacle, and the contact frame is the end-effector pose of the expert's placement. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact stay in the SR denominator.

Table S4. **Baseline training.** All methods train on the same 11,200 demonstrations with the official recipe of each method and run under the shared executor of Appendix B.3. Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM. DreamVLA samples with ten DDIM steps and classifier-free guidance of 1.5 and uses the better of its 40-epoch and a 20-epoch cosine schedule on development scenes; OpenWAM-α generates video and actions jointly with video decoding disabled.

Method	Initialization	Updates×batch	H	Params
ACT [17]	scratch	50,000 × 32	100	[X]
DP [1]	scratch	50,000 × 32	8	[X]
$\pi_{0.5}$ [9]	`pi05_base`	30,000 × 256	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	[X]
DynamicVLA [15]	DOM ckpt.	50,000 × 44	20	[X]
PUMA [2]	official	[X]×[X]	[X]	[X]
DreamVLA [16]	released	48,220 × 512	3	[X]
OpenWAM-α [13]	official	34,490 × 192	32	[X]
DynamicWAM [8]	official	40,000 × 128	16	[X]
DynamicWAM+SE(3)	official	40,000 × 128	16	[X]
SE(3)-WAM	scratch	51,740 × 512	8	523.8M

Training seeds and intervals. The full model and the five ablation controls use training seeds 0, 1, and 2, and each baseline uses seed 0. We report each training seed and the cross-seed mean and standard deviation (Appendix C.4), and for the ablations also the comparison without the phase-only wire passage (`t3_phase`). For paired 95% intervals, we average the seeds per scene and bootstrap held-out scenes within each task, jointly across methods. These intervals quantify scene uncertainty for the trained weights, and we treat differences whose interval lies within ±2 points as ties.

B.5. Baselines and Ablation Controls

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S4), trains once with seed 0 on the same 11,200 demonstrations, whose commands we map to its action space through the same Cartesian controller, and is configured on development scenes. All methods share the agent-view camera and receive the further inputs of Table 1; methods whose recipes expect wrist views receive masked or black wrist inputs.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.2 from the same pose source, with the same validity flags and masking. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them; for DynamicWAM, they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 change one component each of the configuration of Table S2 and keep the data, the recipe, the update budget, the executor, and the training seeds 0, 1, and 2. *No target history* removes slots 0–6 of the target-state tokens, which carry the past poses with their velocity and acceleration features, and keeps slot 7 with the current pose (Appendix A.2). *No future supervision* removes $\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the reconstruction of the current target region. *No scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout, and *no dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head; both keep the target-state tokens and all appearance targets. The *uniform-fidelity forecast* replaces $\mathcal{L}_P$ and $\mathcal{L}_F$ by [a forecast of future latents of a frozen video VAE of the whole frame at the same horizons, its readout, and its loss weight]. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

C. Additional Simulation Results

C.1. Per-Task Results

Table S5 breaks the held-out Dynamic-14 results of Table 1 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.4). With its native 100-step DDPM sampler, every chunk of Diffusion Policy has expired when it arrives, so none of its commands is executed. [Tasks on which every method fails or a baseline outperforms SE(3)-WAM, and the dominant failure of SE(3)-WAM.]

C.2. Evaluation Protocols and Latency

Table S6 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of SE(3)-WAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between SE(3)-WAM and [name] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S7 gives the success at every added latency; its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

Timing. Table S8 breaks the time-to-action of SE(3)-WAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S4 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

C.3. Results on DOMINO

On DOMINO [2], we compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). SE(3)-WAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.2. The RGB-based baselines do not receive this pose. [Summarize the DOMINO results.]

C.4. Final Ablations and Seed Stability

Table S10 reports the five controls of Sec. 4.3 (Appendix B.5) for each training seed, with the cross-seed mean and standard deviation and the paired difference from the full model, with native latency and with 150 ms of added latency. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`). On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR, mostly on this task, whose success ranged from [0] to [71]% across seeds (Appendix B.4). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes for an individual seed or without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency and

Table S5. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: SE(3)-WAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.4); –: no 13-task mean. Task codes are listed in Table S3.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	48.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	46.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	18.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	21.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	32.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	26.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage	99.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture	36.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	71.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Slider capture	57.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Can capture	90.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	44.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	42.0	0.0	—	—	—	—	—	—	—	—	—	—	—
e_p (mm)	32.2	–	—	—	—	—	—	—	—	—	—	—	–
n	1109	6	—	—	—	—	—	—	—	—	—	—	–

Table S6. **Protocol controls** on held-out Dynamic-14 scenes (SR, %) under the protocols of Appendix B.3; real time is the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.9	—	—	42.0	0.0	50
Diffusion Policy	—	—	—	0.0	100.0	–
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM	—	—	–	—	—	—

Table S7. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	42.0	—	—	24.1	—
Diffusion Policy	0.0	—	—	0.0	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—

with all three training seeds. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise

x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of SE(3)-WAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of SE(3)-WAM under synthetic perturbations of the target pose, relative to clean poses.

Table S8. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

Table S9. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

Table S10. **Final ablations by training seed** on held-out Dynamic-14 scenes (Appendix C.4). S0–S2: SR (%) of training seeds 0–2; mean ± s.d. over seeds; Δ: control minus SE(3)-WAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks.

	Native					+150 ms					Δ w/o `t3_phase`	
Variant	S0	S1	S2	Mean ± s.d.	Δ [95%]	S0	S1	S2	Mean ± s.d.	Δ [95%]	Native	+150 ms
SE(3)-WAM ⋆	—	—	—	—	–	—	—	—	—	–	–	–
No target history	—	—	—	—	—	—	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—	—	—	—	—	—	—
No dedicated SE(3)	—	—	—	—	—	—	—	—	—	—	—	—
Uniform-fidelity forecast	—	—	—	—	—	—	—	—	—	—	—	—

C.6. Forecast Diagnostics

Table S9 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

D. Real-Robot Experiments

D.1. Setup and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. SE(3)-WAM uses this fixed third-person view; baselines whose recipes require a wrist view receive [per-method camera configuration]. All policies run on one RTX 5090. Fig. S2 shows the layout, and Table S11 defines the six tasks. T1 and T2 share a belt conveyor, T3 uses a turntable, T4 a remotely driven toy car, T5 a cue-driven collision between two balls, and T6 a ramp. Touching or pushing the target without completing the task, a single-finger hold, and an action after the target has left the workspace count as failures; [equipment-invalid trials and safety interventions are repeated or counted as failures by the following rule].

D.2. Pose Tracking and Demonstration Labels

Tracking is marker-free. Grounding DINO [7] localizes the object named by the task in the first frame, SAM 2.1 [10] gives its initial mask, and FoundationPose [14] initializes the object's 6D pose from this mask and then tracks it. [FoundationPose variant and mode (model-based with an object mesh or model-free with reference views), how the object frame is kept consistent across demonstrations and trials, and re-initialization after a loss.] The tracker runs on the control clock: at every control-period boundary, it updates the pose on the newest camera frame captured at or before the boundary, so the pose history has the 50 ms spacing used in training, and each estimate keeps the capture time of its frame. A query waits for the update of its own boundary and then runs the policy on the same GPU. While a query spans further boundaries, the tracker keeps updating at a higher GPU priority and processes only the newest frame, so that an update never queues behind older ones; the tracking time and this contention are part of the measured latency (Table S8). [Camera frame rate and the measured time per update.] When the target is lost, the estimate is marked invalid, and the previous pose is not carried forward. [How the target poses of the demonstrations, used as inputs and training labels, are obtained.] For the ball tasks, the target-state tokens keep the translation, velocity, and acceleration of the ball center and mask its orientation and angular motion, and

Figure placeholder: Fig. S2 — real-robot setup

(a) Left, photo of the workcell from behind the robot: the third-person RGB-D camera, the robot, and the conveyor, turntable, car track, ball-collision track, and ramp at their operating positions, each labeled; the wrist camera if a baseline uses one. **(b) Right, top-down schematic to scale:** the robot base frame with its axes, the camera position and viewing direction with its distance to the workspace, the footprint and motion direction of each device for T1–T6, the grasp or interception region of each task, and the workspace bounds in cm. **Rules:** real photo, unretouched; schematic from the calibrated extrinsics and measured device positions; no repetition of the task photos of Fig. 6.

Figure S2. **Real-robot setup.** (a) The workcell. (b) Top-down layout of the camera, the robot base frame, and the devices of T1–T6 [with the workspace dimensions].

Table S11. Real-robot task definitions. $N = 100$: training demonstrations per task; 20 additional demonstrations per task are reserved for validation.

Task	Device, motion, and objects	Operating range; N	Success and failure criteria
T1	Conveyor transports a graspable cube.	[belt speed]; 100	[stable grasp and lift; failures]
T2	Conveyor transports an open box; the arm places a cube into it.	[belt speed]; 100	[placement and retention; failures]
T3	Turntable carries a cube around its axis.	[angular speed, radius]; 100	[stable grasp and lift; failures]
T4	Remotely driven toy car moves on the tabletop.	[speed, path]; 100	[car grasp and lift; failures]
T5	A cue-driven ball strikes a second ball; the arm intercepts the second ball after the collision.	[impact and outgoing speed]; 100	[post-impact capture; failures]
T6	A ball rolls or a block slides down a ramp toward the interception region.	[slope, release, speed]; 100	[capture and retention; failures]

Table S12. **Real-robot pose tracking.** Median/p95 translation (mm) and rotation (°) error of the tracker against a marker-free reference, the uncertainty of that reference, and the fraction of lost observations. Ball rotation is not used.

Target	mm	°	Ref. uncertainty	Lost
Real rigid object	—	—	—	—
Real ball	—	–	—	—

the pose head is supervised on translation only. Table S12 reports the tracking accuracy.

D.3. Training and Trial Protocol

We compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [1], with 20 trials per task and method. All methods use the same pre-generated trial keys, which fix the object instance, the initial pose, and the motion parameters, sampled by [sampling protocol]. Trials run in [randomized method order], so that drift of the apparatus does not align with the methods. An operator resets the scene. A trial starts at the first usable observation, the robot may move at any time, and a trial ends at success, failure, or a timeout of [X] s. Demonstrations, development trials, and test trials come from separate sessions. SE(3)-WAM is fine-tuned per task from the DROID base (Appendix A.4), and the baselines are trained or fine-tuned on the same demonstrations with [per-method initialization and training settings]. For every method, we log the end-to-end time-to-action of each query on the same RTX 5090, including tracking for SE(3)-WAM; Table 4 reports its p95.

D.4. Detailed Results and Failures

Table S13 reports successes per task, 95% binomial intervals for the pooled rates, and paired differences from a bootstrap over trial keys. [The most frequent failures of SE(3)-WAM and their causes.]

Table S13. **Real-robot successes** per task (successes/trials, 20 trials per task; tasks in Table S11). All: pooled success rate (%) with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	–

E. Preliminary Design Studies

We screened the encoder, the history representation, the forecast targets, and the coupling of world and action streams with smaller models: 4 layers of width 768, trained for 4 epochs (40,104 updates at a global batch size of 16) and evaluated with the synchronous executor that preceded our real-time protocol. Each group keeps its own encoder and seed count, stated below, and conditions are compared within a group. These studies selected the components of the final model; the five controls of Sec. 4.3 then test its central components at full scale and under the real-time protocol (Appendix C.4).

E.1. Encoder, Backbone, and History

Encoder and backbone. We compared a CNN trained with the policy, frozen DINOv3 ViT-S/16 and ViT-L/16 [11], and MAE ViT-B/16 [4] (three seeds for the MAE reference; Table S14), and selected MAE. We also compared a backbone with shared attention and stream-specific feed-forward networks against one with fully separate world and action parameters, at about 47.3M backbone parameters and with three seeds each ([source of the backbone comparison]), and selected the shared attention.

History. With a frozen DINO-L encoder and one seed per condition, we compared the current state alone, the target-motion history, RGB windows, Farnebäck flow [3], flow with motion statistics [8], codec motion vectors, video-VAE latents, and compressed windows (Table S15). Its last row replaces the past target poses of the trained history model by its current pose at test time.

E.2. Forecast Targets and Auxiliary Supervision

With a frozen DINO-L encoder and one seed per condition, we compared fifteen future targets, from target-only and image-space forecasts to role-aware combinations (Table S16). A second group with the MAE encoder and one seed removed each future appearance target from the selected configuration. In the first group, no future target improved clearly on the variant without future supervision (P00, 62.78% against 62.38% for scene flow and SE(3)); single seeds at this scale do not resolve such differences, and the full-scale ablations of Sec. 4.3 test future supervision in the final model.

Table S14. **Observation encoders** at the screening scale (Appendix E.1). Params: encoder parameters; p95: time-to-action.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16*	86M	frozen	—	—	—

Table S15. **History representations** at the screening scale (frozen DINO-L encoder, one training seed, 1,400 scenes). The first eight conditions receive the current target pose; the ninth adds the target-motion history. Last row: the ninth model with its past target poses replaced by the current pose at test time and the kinematic features recomputed.

History input	Success / 1,400	SR (%)
Current image	748	53.43
RGB window	662	47.29
Optical flow	720	51.43
Flow + motion statistics	723	51.64
Codec motion vectors	733	52.36
Video-VAE latent	710	50.71
Compressed window (age)	634	45.29
Compressed window (uniform)	670	47.86
Current image + target-motion history	824	58.86
stationary history at test time	9	0.64

E.3. World–Action Coupling

With the MAE encoder and three seeds per condition, we compared six couplings of the world and action streams. R0 disables every forecast loss. R1 adds one object query and one query per patch to the world stream, which attend to $\mathcal{S}$, $\mathcal{M}$, and each other and carry the forecast readouts, without any edge to the actions. R2 lets the action tokens attend to these queries at every layer, R3 lets the queries attend to the actions, and R4 opens both edges; in R3 and R4, the world states are recomputed at every Euler step. R5 decodes the forecast once per query and encodes the target trajectory and the flow with visibility, with rotations in a continuous 6D form, through a geometry encoder (0.37M parameters) into tokens that the actions attend to. Future supervision without

Table S16. **Forecast targets** at the screening scale. The first group uses a frozen DINO-L encoder and seed 0, with its 140 development scenes and the 1,260 remaining scenes reported separately. The second group uses the MAE encoder and seed 1 on 1,400 scenes; it keeps scene flow, SE(3), and the current-target RGB reconstruction and removes future appearance targets. Compare conditions within each group. Targets are cumulative from the current state at the eight horizons, except codec motion: SE(3), the target pose; 3D scene flow, base-frame displacements of the 14×14 patch centers; 2D tracks and dense 2D displacement, image displacements of target points, grid points, or all pixels; codec motion, the motion vectors of a video codec; video-VAE latent, latents of a frozen Wan2.2 VAE; object DINO, mask-pooled DINOv3 features of the target.

ID	Future supervision	Dev. success / 140	Success / 1,260	SR_{1260} (%)
P00	None	88	791	62.78
P01	3D scene flow	95	782	62.06
P02	SE(3)	92	789	62.62
P03	Scene flow + SE(3)	93	786	62.38
P04	Video-VAE latent + SE(3)	92	783	62.14
P05	Dense 2D displacement + SE(3)	87	746	59.21
P06	Codec motion + SE(3)	86	795	63.10
P07	Sparse scene 2D tracks + SE(3)	90	807	64.05
P08	Scene flow + object DINO	89	794	63.02
P09	Scene flow + target 2D tracks	88	808	64.13
A_α	P03 + motion magnitude	89	778	61.75
P12	P03 + depth	90	791	62.78
P13	P03 + DINO	92	788	62.54
P14	P03 + dynamic RGB	89	790	62.70
P15	P03 + depth / DINO / dynamic RGB	94	802	63.65
AUX0 MAE-B: future appearance auxiliaries		Success / 1,400		SR_{1400} (%)
None		894		63.86
Depth only		867		61.93
DINO only		946		67.57
Dynamic RGB only		926		66.14
All three (existing reference)		995		71.07

Table S17. **Couplings R0–R5** at the screening scale (MAE encoder, three training seeds each, 1,400 development scenes per seed; Appendix E.3). SR: mean over seeds (%); s.d.: standard deviation over seeds (points).

Coupling	SR	s.d.
R0 (future losses off)	60.45	3.57
R1 (queries, no edge)	63.60	2.34
R2 (world→action)	62.52	1.97
R3 (action→world)	64.21	2.13
R4 (bidirectional)	63.00	4.19
R5 (decoded geometry)	63.95	1.91

any edge to the actions (R1) exceeded R0 by 3.14 points on average, with seed-wise differences of −1.29, +9.64, and +1.07. Feeding the forecasts to the actions gave no consistent further gain: R2 and R5 differed from R1 by −1.07 and +0.36 points on average, with standard deviations of 4.28 and 3.45 points over the seed-wise differences, and the effects of both cross-stream edges changed sign across seeds (Table S17). We therefore use the forecasts only as training supervision, which also keeps deployment free of their cost.

References

[1] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 5, 9

[2] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 5, 6

[3] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 10

[4] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1, 10

[5] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2

[6] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[7] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 8

[8] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 5, 10

[9] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 5

[10] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 8

[11] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 2, 10

[12] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 3

[13] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 5

[14] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8

[15] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 5

[16] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xin-Qiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 5

[17] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 5

[18] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 3